Week 3 — Hackathon Pack

Citizen-Facing Service Prototype: the Green Home Grant

Over two days your team designs, builds, tests and presents a digital service for the Green Home Grant, a fictional government scheme that funds home insulation and heat pumps. Today citizens phone a call centre and wait an average of 35 minutes to find out whether they qualify. Your service replaces that call.

This is your first assessed portfolio piece. Use this pack alongside the brief and the lab README: it takes you through setup, each block of the build, the checkpoints and the presentation.

The habit to hold on to: **Claude is your intern developer.** Read and check every change it makes before you commit it, and log what you changed.

BEFORE YOU START

- **Open VS Code** at your `code-labNXXX` address, and open a terminal.
  - Your lab number (the NXXX in every address) is in your welcome email.
- **Download the brief** PDF before you read it.
  - It does not always display properly if you preview it in the browser.
- **Work only in** `~/Documents/my-work/`.
  - Copy the starter from `readonly-courseware/`.
  - Do not run code from `readwrite-classroom/`: it is shared, has no version history, and programs cannot be run from it.
- **Agree how your team shares code** before you start building.
  - If everyone can reach your Git host: one shared repository, one branch per person or page.
  - If not: one person's machine is the "driver", and the others review and pair.
- **Your app** opens at `https://<hostname>:5002`.
  - Run `hostname` in the terminal to find your lab machine's name, and use `https`, not `http`.
  - The `localhost` address Vite prints only works inside the lab machine, not in your browser.
- **Blocked?** Say so in chat straight away.
  - A facilitator will come to your room. Nothing else matters until your app runs.

Activities

PART 1: THE BRIEF

1 Read the brief and agree your scope
2 Know your checkpoints

PART 2: DAY 1 — BUILD

3 Set up and see your app
4 Plan the architecture
5 Build the start page and first questions
6 Complete the flow and the eligibility logic
7 Day 1 checkpoint
8 Fix, commit and plan Day 2

PART 3: DAY 2 — POLISH, TEST, PRESENT

9 Check your answers and validation
10 Accessibility and 320px

11 Unit test your eligibility rules
12 Service Standard signals
13 Prepare and give your presentation
14 Peer feedback
15 Retrospective and submit

APPENDIX

— Before you finish: checklist
— Glossary

Part 1: The brief

What your client needs, the floor every team must ship, and how the work is judged.

Activity 1 Read the brief and agree your scope

Brief

In this activity, you'll read the brief with your team and agree what you will build first. By the end, you'll be able to name the six items every team must ship and explain how your work will be judged.

TO GET STARTED

1. Read the brief: the problem, the requirements, the Minimum Viable Submission (MVS) and the rubric.
2. With your team, agree who owns which pages, how you will share code, and when you will check in with each other.
3. Go through the six MVS items below. Make sure everyone understands each one before anyone mentions stretch goals.
4. Note the checkpoint times in the second table below. Plan backwards from them.

#	Minimum Viable Submission: ship all six first
1	**Multi-page question flow:** a start page and at least 5 question pages, one question per page.
2	**Check your answers:** a summary list with a Change link for every answer.
3	**Confirmation page:** a clear outcome: eligible, not eligible or partially eligible.
4	**GOV.UK components:** typography, spacing and headings from the provided CSS variables.
5	**WCAG 2.2 AA:** labelled inputs, keyboard use, 4.5:1 contrast, visible focus.
6	**AI usage log:** `AI_LOG.md` with at least 3 entries, four fields each.

CHECK YOU ARE DONE

Your team can say the six MVS items from memory, and everyone knows which pages they own.

HINTS

1. **Where are the eligibility rules?** The brief does not give them. Your team decides them (for example income bands, owner or renter, what counts as already insulated), writes them down and builds them. That decision is part of the work.
2. **What does "seeded" AI log mean?** Only that `AI_LOG.md` is already in your starter folder, with one example entry.

Stretch: Read the rubric's Excellent column and agree, as a team, which row you most want to excel at.

Activity 2 Know your checkpoints

Plan of action

In this activity, you'll note the moments facilitators will check in and what to show at each. By the end, you'll be able to plan your team's two days backwards from them.

TO GET STARTED

1. Read the table below with your team.
2. Agree what you want to have working by each time.

When	What you should be able to show
Day 1, 12:30	**First green path:** one route from start page to result page renders, however rough.
Day 1, 14:30	**Day 1 checkpoint:** Start now → questions → result, with Continue, Back and saved answers.
Day 2, 10:50	**Mid-morning check:** check your answers renders, one validation error works, 2+ AI log entries.
Day 2, 13:30	**Presentation:** a 5-minute live demo.
Day 2, 16:00	**Final submission:** repository link or zip, meeting the checklist at the end of this pack.

CHECK YOU ARE DONE

Your team has written down its own target for each checkpoint.

Part 2: Day 1 — Build

Get the starter running, plan the architecture, then build the start page, the question flow and the eligibility logic.

Activity 3 Set up and see your app

Lab README: Setup

In this activity, you'll copy the starter, install it and open it in your browser. By the end, you'll be able to run the development server and see your changes in the browser.

TO GET STARTED

1. Copy the starter into a new folder. Type each line and press `Enter` after it. The last command is split over two lines: type it as one line, without the `\`.

```
mkdir -p ~/Documents/my-work/w03-hackathon
cd ~/Documents/my-work/w03-hackathon
cp -r ~/Documents/readonly-courseware/swe-v1-ukds-c1/courseware/\
labs/w03-hackathon-citizen-service/hackathon/starter/. .
```

2. Install the dependencies and start the development server. The install takes up to a minute.

```
npm install
npm run dev
```

3. Find your lab machine's name, then open `https://<that name>:5002` in your browser. You should see the start page heading.

```
hostname
```

4. Leave this terminal running. Open a second terminal for Claude Code: **Terminal → New Terminal**, then `cd ~/Documents/my-work/w03-hackathon` and `claude`.

CHECK YOU ARE DONE

The starter's start page loads in your browser, and the development server is still running in its own terminal.

HINTS

1. **"Port 5002 is already in use"?** Something else is using that port, maybe a teammate's server or an earlier run. Stop it with `Ctrl+C` in its terminal, or pick another port from 5000 to 5005, for example `VITE_PORT=5003 npm run dev`, and change the port in the address.
2. **The page keeps reloading or does not update?** Check you are using the `https://<hostname>:5002` address, not `localhost`. The starter's `vite.config.js` is already set up for the lab; do not let Claude change the `server` section.
3. `npm: command not found`**?** Close the terminal, open a new one and try again. If it still fails, say so in chat.

Stretch: Ask Claude to explain `vite.config.js` line by line. Log it if you learned something.

Activity 4 Plan the architecture

Brief: Day 1

In this activity, you'll plan the service with your team before building it, using AI to help. By the end, you'll be able to describe your component tree, your form state and your eligibility rules.

TO GET STARTED

1. Sketch the component tree on paper or a whiteboard: start page → five question pages → check your answers → result, plus header, phase banner, footer and accessibility statement.

2. Agree one state object held in `App`, with a field per question: `propertyType`, `ownership`, `incomeBand`, `insulation`, `heating`.
3. Decide your eligibility rules. Fill in the table below, then write the rules in a short file, for example `docs/eligibility-rules.md`.
4. Ask Claude, in plan mode (`Shift+Tab`), for an architecture or pseudocode based on your sketch. Review it as a team and change what you disagree with.
5. Record this as your first `AI_LOG.md` entry: what you asked, what Claude produced, what you changed and why.
6. Optional: add a `CLAUDE.md` telling Claude the stack, the GOV.UK class names in `App.css` and your team's conventions.

Question	Answers you will offer	Effect on eligibility
Property type		
Ownership		
Household income band		
Existing insulation		
Current heating		

CHECK YOU ARE DONE

Your team can show its component tree, state object and written eligibility rules, and `AI_LOG.md` has a planning entry.

HINTS

1. **Cannot agree on the architecture?** Start with the start page and build one question at a time. The routes are already in `App.jsx`: follow them.
2. **Should we plan first or just start?** Both approaches work. Choose one on purpose, and say which you chose when you present.

Stretch: Turn your rules table into a list of test cases now (these answers → this result). You will use them for unit tests on Day 2.

Activity 5 Build the start page and first questions

Lab README: Acceptance criteria

In this activity, you'll build the GOV.UK start page and the first three question pages, wired to shared state. By the end, you'll have a first green path: one route from start to result that renders.

TO GET STARTED

1. Build the start page: a title, a short description and a "Start now" button that goes to the first question.
2. Build `PropertyTypePage`, `OwnershipPage` and `IncomePage`: one question per page, radio buttons each with a `<label>`, a Continue button and a Back link.
3. Make Continue save the answer to the state in `App`, then go to the next page. Make Back return to the previous page.
4. Let Claude scaffold each page, then read the code before you commit it. Log anything you changed.
5. By 12:30, make sure one path reaches the result page, even if the later pages are placeholders.

CHECK YOU ARE DONE

Start now leads to three working questions; your answers are still selected when you go Back; one path reaches the result page.

HINTS

1. **Answers disappear when you click Back?** The state is probably inside each page. Lift it into `App` as one `useState` object and pass the value and the setter to each page.
2. **How do we move between pages?** Use `useNavigate` from react-router-dom: Continue saves, then calls `navigate("/next-page")`; Back calls `navigate(-1)`.
3. **The styling looks wrong?** Use the GOV.UK classes already defined in `App.css`: `govuk-heading-xl`, `govuk-body`, `govuk-button`, `govuk-form-group`, `govuk-label`, `govuk-radios`. You should need almost no custom CSS.

Stretch: Mount the phase banner and footer now (see the `TODO` comments in `App.jsx`).

Activity 6 Complete the flow and the eligibility logic

Lab README: Hints

In this activity, you'll finish the question flow and write your eligibility rules as a function. By the end, you'll have a result page that shows an outcome worked out from the user's answers.

TO GET STARTED

1. Build `InsulationPage` and `HeatingPage` the same way as the first three.
2. Write your eligibility rules as a **pure function** in its own file, for example `src/utils/eligibility.js`. It takes the state object and returns the outcome (eligible, not eligible or partial) and a reason. It does not touch the page.
3. Make `ResultPage` call the function and show the outcome and the reason.
4. Pair up: one person drives, one reviews what Claude suggests before it is accepted.

CHECK YOU ARE DONE

All five questions work with Continue and Back, and the result page shows an outcome worked out by your function.

HINTS

1. **The rules are spread across the pages?** Move them into the one function. Pages collect answers; the function decides. That also makes it testable on Day 2.
2. **Not sure what "pure" means?** The function only uses what it is given and only returns a result. Same answers in, same result out, every time.

Stretch: Your floor path works? Start check your answers, or add save-and-return with `localStorage`.

Activity 7 Day 1 checkpoint

Plan of action: 14:30

In this activity, you'll demo your progress to the facilitators in about three minutes. By the end, you'll have one piece of feedback to act on.

TO GET STARTED

1. Choose who will share their screen, and have the app open before your turn.
2. Show: the start page and Start now; the question pages going forward and Back; your answers still there after navigating; one complete path to a result.
3. Write down the feedback you are given.

CHECK YOU ARE DONE

You have shown one complete path and written down one piece of feedback.

Activity 8 Fix, commit and plan Day 2

Lab README: Day 1 target

In this activity, you'll act on your checkpoint feedback and leave Day 1 in a working state. By the end, you'll have a commit to come back to and a list of Day 2 priorities.

TO GET STARTED

1. Fix the blocker from your checkpoint feedback first.
2. Start the check your answers page if you have not already.
3. Commit your working state.
4. Bring `AI_LOG.md` up to at least two entries.
5. Write a short list of your Day 2 priorities.

CHECK YOU ARE DONE

Your working state is committed, `AI_LOG.md` has at least two entries, and you have written your Day 2 priorities.

Part 3: Day 2 — Polish, test, present

Finish the floor, prove it works for everyone, test the rules, then present, review and reflect.

Activity 9 Check your answers and validation

Brief: Day 2

In this activity, you'll build the check your answers page and add validation to every question. By the end, a user will be able to review and change their answers and will see clear errors if they miss one.

TO GET STARTED

1. Build `CheckAnswersPage`: a GOV.UK summary list showing every answer.
2. Add a "Change" link to each answer that returns to that question with the answer still selected.
3. Add validation to every question: if nothing is chosen, show an **error summary** at the top of the page and an **inline error** on the question.
4. Use Claude to write the validation logic, then read it. Record it in `AI_LOG.md`.

CHECK YOU ARE DONE

Every answer appears on the summary with a working Change link, and submitting an empty question shows both the error summary and the inline error.

HINTS

1. **The error messages are vague?** Claude often writes "This field is required". GOV.UK messages say what to do, for example "Select the type of property you live in".
2. **Change loses the other answers?** The answers must live in the state in `App`, not in the page.
3. **Did Claude build the summary as a table?** AI often does. The GOV.UK summary list is a `<dl>` (description list) with `govuk-summary-list` classes, and each Change link needs hidden text such as "Change property type" for screen readers.
4. **Is the error summary just a styled box?** It needs `role="alert"` (not `alertdialog`), the heading "There is a problem", and a link from each error to the field it belongs to.

Stretch: After a Change, return the user to the summary instead of the next question.

Activity 10 Accessibility and 320px

WCAG 2.2 AA

In this activity, you'll test your service the way a disabled user or an assessor would, and fix what fails. By the end, you'll be able to test for keyboard use, labels, contrast and reflow.

TO GET STARTED

1. **Keyboard:** put the mouse aside. Use `Tab` to move, `Enter` or `Space` to choose. Complete the whole journey. Focus must always be visible.
2. **Labels:** every radio button and input has a `<label>`. Clicking the label should select the option.
3. **Contrast:** right-click some text, choose **Inspect**, and click the colour swatch in the Styles panel to see the contrast ratio. It must be at least 4.5:1.
4. **320px:** in DevTools, turn on the device toolbar (`Ctrl+Shift+M`) and set the width to 320. Go through every page: nothing should scroll sideways.
5. **Screen reader spot-check:** use VoiceOver or NVDA if you have one, or the Accessibility pane in DevTools.

CHECK YOU ARE DONE

You can complete the whole journey with the keyboard alone, every input has a label, text passes 4.5:1, and nothing scrolls sideways at 320px.

HINTS

1. **Short on time?** Do the keyboard and label checks first. They find the most problems and count most in the rubric.
2. **Claude left out ARIA?** It often does unless asked. Ask it specifically, then check the result with the keyboard test.

Stretch: Add Playwright end-to-end tests that run the keyboard and 320px checks automatically.

Activity 11 Unit test your eligibility rules

Requirement 9

In this activity, you'll add a test runner and prove your eligibility function with at least five tests. By the end, you'll be able to run your tests with one command and see them pass.

TO GET STARTED

1. The starter has no test runner. Install Vitest version 1, which works with the starter's version of Vite:

   ```
   npm install -D vitest@1
   ```

2. Open `package.json` and add `"test": "vitest run"` to the `"scripts"` section.
3. Create a test file next to your function, for example `src/utils/eligibility.test.js`. Here is the shape of one test. Change the function name, the answers and the expected result to match your own rules.

   ```
   import { describe, it, expect } from 'vitest';
   import { checkEligibility } from './eligibility';

   describe('checkEligibility', () => {
     it('says council tenants are not eligible', () => {
       const result = checkEligibility({ ownership: 'rent-council' });
       expect(result.status).toBe('not-eligible');
     });
   });
   ```

4. Write at least five tests: at least one each for eligible, not eligible and partially eligible, plus your trickiest rule.
5. Run them:

   ```
   npm test
   ```

CHECK YOU ARE DONE

`npm test` shows five or more passing tests.

HINTS

1. **Nothing to test because the rules are in the pages?** Move them into the pure function first (Day 1, Activity 6).
2. **Let Claude write the tests?** You can, and log it. But write the expected results yourself: you decide what "correct" means, not Claude.

Stretch: Test first: write a test for a new rule, watch it fail, then ask Claude to make it pass.

Activity 12 Service Standard signals

Rubric: Service Standard

In this activity, you'll add the signs a real GOV.UK service is expected to show. By the end, your service will have an accessibility statement, a phase banner, a footer and a cookie notice.

TO GET STARTED

1. Mount the phase banner under the header, showing "Alpha" and a feedback link.
2. Mount the footer, with a link to the accessibility statement.
3. Fill in `AccessibilityStatementPage` using what you found in your accessibility tests.
4. Add a simple cookie notice.

CHECK YOU ARE DONE

The phase banner, footer, accessibility statement and cookie notice all appear and can be reached with the keyboard.

Activity 13 Prepare and give your presentation

Plan of action: presentation

In this activity, you'll demo your service live in five minutes. By the end, the cohort will have seen your service working and heard how your team worked.

TO GET STARTED

1. Decide who presents and who drives the screen, and have the app open before your turn.
2. Rehearse the order: (1) an eligible user, end to end; (2) a not-eligible path; (3) one validation error; (4) one `AI_LOG.md` entry showing what the AI did and what you changed; (5) your tests passing.
3. Keep to five minutes; no slides are needed. Then take one or two questions.

CHECK YOU ARE DONE

You have shown all five things within five minutes.

Activity 14 Peer feedback

Rubric

In this activity, you'll review two other teams' services against the rubric. By the end, you'll have given and received specific, written feedback.

TO GET STARTED

1. Open the two services you have been given. For each rubric row, write one strength and one improvement.
2. Check in particular: does it meet each requirement, is it keyboard-navigable, are the error messages clear?
3. Share your written feedback with the team.

CHECK YOU ARE DONE

Each team you reviewed has one strength and one improvement for every rubric row.

Activity 15 Retrospective and submit

Brief: Recap

In this activity, you'll reflect on how your team worked with AI, then submit. By the end, you'll have three to five takeaways to use in your own work.

TO GET STARTED

1. Join the whole-cohort discussion: which workflow helped most, where AI code needed most correction, the hardest GOV.UK pattern, what you would do with a third day.

2. Write three to five takeaways for your own work, and share one in the chat.
3. Make your final commits, check `AI_LOG.md` has at least three entries, and submit your repository link or a zip.
4. Complete the course feedback form.

CHECK YOU ARE DONE

Your submission is in and meets the checklist on the next page.

Before you finish: checklist

The service

- ☐ A GOV.UK start page with a title, a description and a "Start now" button.
- ☐ At least 5 question pages, one question per page.
- ☐ Continue and Back on every question page.
- ☐ A check your answers page with a Change link for every answer.
- ☐ A result page: eligible, not eligible or partially eligible.

Accessibility and design

- ☐ Every form input has a `<label>`.
- ☐ Keyboard navigation works (`Tab`, `Enter`, `Space`), with visible focus.
- ☐ All text has at least 4.5:1 contrast.
- ☐ The layout reflows at 320px with no sideways scrolling.
- ☐ Validation uses the GOV.UK error pattern: error summary plus inline error.

Evidence

- ☐ 5 or more unit tests pass for the eligibility logic.
- ☐ `AI_LOG.md` has 3 or more entries, four fields each.
- ☐ The 5-minute presentation has been given.
- ☐ Your repository link or zip has been submitted.

Glossary

Term	What it means
Accessibility statement	A page public sector services must publish, saying how accessible the service is and how to report problems.
`AI_LOG.md`	Your team's record of AI-assisted work: task, what the AI generated, what you changed and why.
Check your answers	A GOV.UK page that lists every answer with a Change link before the user confirms.
`CLAUDE.md`	A file of instructions Claude Code reads at the start of every session in that folder.
Contrast ratio	How different the text colour is from its background. 4.5:1 is the minimum for normal text.
Error summary	A box at the top of a page listing every error, each linking to the question it belongs to.
GOV.UK Design System	The official styles, components and patterns for UK government services.
`localStorage`	Storage in the browser that keeps data after the page is closed, used for save-and-return.
MVS	Minimum Viable Submission: the six items every team must ship before anything else.
npm	The tool that installs JavaScript packages (`npm install`) and runs project scripts (`npm run dev`, `npm test`).
One thing per page	The GOV.UK pattern of asking one question on each page.
Phase banner	A strip under the header saying the service is in alpha or beta, with a feedback link.
Playwright	A tool that drives a real browser to test a whole journey automatically.
Pure function	A function that only uses what it is given and only returns a result, so the same input always gives the same output.
React component	A reusable piece of a page, written as a JavaScript function that returns what to show.
Reflow	Content rearranging to fit a narrow screen without sideways scrolling.
Route	A web address inside the app, such as `/income`, that shows a particular page.
Start page	The first page of a GOV.UK service: what it does, and a "Start now" button.
State	Data a React app remembers while it runs, such as the user's answers. `useState` creates it.
Unit test	A small piece of code that checks one function gives the expected result.
Vite, dev server	The tool that runs your app while you build it, and reloads the page when you save.
Vitest	A test runner for Vite projects. `npm test` runs your tests with it.
WCAG 2.2 AA	The Web Content Accessibility Guidelines, level AA: the accessibility standard UK public sector services must meet.